

Redirecionamento Logado Para Sites Externos

Guia de montagem de URL — LoaderScreenGenericRedirect

O que é essa rota

`LoaderScreenGenericRedirect` é uma tela "invisível" do app que:

1. Recebe parâmetros via deep link (query string).
2. Descobre, a partir do parâmetro ``type``, qual é o **domínio/URL base** de destino.
3. Monta a URL final juntando: ``URL base` + `path` + `parâmetros extras``.
4. Abre essa URL final **dentro do app** (WebView) ou **no navegador externo**, dependendo do parâmetro ``external``.

 Se qualquer regra abaixo não for respeitada, o usuário cai numa **tela de erro genérica** ("Não conseguimos concluir o seu redirecionamento").

Formatos de link aceitos

Toda URL de destino pode ser criada em dois formatos equivalentes:

Formato	Uso	Exemplo
Deep link nativo (custom scheme)	Abre o app diretamente, se instalado (usado dentro do próprio app, notificações push, etc.)	<code>`minhaclaro://LoaderScreenGenericRedirect?type=residencial&path=minhas-assinaturas`</code>
Smart link / link universal (https)	Usado em canais externos (SMS, e-mail, WhatsApp, campanhas), funciona mesmo sem o app instalado	<code>`https://app-minhaclaro.clarobrasil.mobi/?deeplink=LoaderScreenGenericRedirect?type=residencial&path=minhas-assinaturas`</code>

Nos dois formatos os **parâmetros e regras são exatamente os mesmos** — a única diferença é o envelope (``minhaclaro://LoaderScreenGenericRedirect?...`` vs ``https://app-minhaclaro.clarobrasil.mobi/?deeplink=LoaderScreenGenericRedirect?...``).

⚠️ ⚠️ Repare que, no formato de link universal, o `?` que separa `LoaderScreenGenericRedirect` dos seus próprios parâmetros **não é escapado** (não vira `%3F`). Siga sempre o padrão dos exemplos existentes.

⚙️ Parâmetros de configuração

Estes são os parâmetros que **controlam o comportamento da tela**. Qualquer outro parâmetro enviado na URL que não esteja nesta lista é tratado como "parâmetro extra" e simplesmente repassado para a URL final (ver seção 5).

Parâmetro	Tipo	Obrigatório	Valor padrão se omitido	Descrição
`type`	string (ver tabela da seção 4)	Sim	— (sem `type` válido, cai direto na tela de erro)	Define para qual domínio/servico o usuário será redirecionado. Não faz diferença entre maiúsculas/minúsculas (`residencial`, `RESIDENCIAL`, `Residencial` funcionam igual).
`path`	string	Não	`` (string vazia)	Caminho que é concatenado logo após a URL base do `type`. Não deve

				começar com <code>`/`</code> (ver regra de concatenação na seção 6).
<code>`external`</code>	boolean (<code>`true`/`false`</code>)	Não	<code>`false`</code>	Se <code>`true`</code> , a URL final abre no navegador externo do dispositivo. Se <code>`false`</code> / omitido, abre em WebView dentro do app .
<code>`tokenAA`</code>	boolean (<code>`true`/`false`</code>)	Não	<code>`true`</code>	Se <code>`true`</code> , injeta o cookie de autenticação "Token AA" na WebView. Só tem efeito quando <code>`external` = false`</code> .
<code>`tokenPing`</code>	boolean (<code>`true`/`false`</code>)	Não	<code>`false`</code>	Se <code>`true`</code> , injeta o cookie de autenticação "Token Ping" na WebView. Só tem efeito

				quando <code>`external`</code> <code>=false`</code> .
<code>`contractSession`</code>	boolean (<code>`true`/`false`</code>)	Não	<code>`false`</code>	Se <code>`true`</code> , injeta o cookie de sessão de contrato na WebView. Só tem efeito quando <code>`external`</code> <code>=false`</code> .

 Os valores booleanos devem ser enviados como texto ``true`/`false`` na query string. Qualquer valor que não seja literalmente esse booleano é ignorado e o padrão é aplicado.

 Valores aceitos para ``type`` e para onde cada um redireciona

<code>`type`</code>	URL base de destino	Precisa de usuário logado?
<code>`residencial`</code>	<code>`https://minhaclaroresidencial.claro.com.br/`</code>	Sim — requer token de autenticação (Token Ping)
<code>`flex`</code>	<code>`https://flex.claro.com.br/`</code>	Sim — requer token de autenticação (Token AA/Flex)
<code>`minha_claro`</code>	<code>`https://minhaclaro.claro.com.br/`</code>	Sim — requer token de autenticação
<code>`claro_tv_mais`</code>	<code>`https://www.clarotvmais.com.br/`</code>	Sim — requer token de autenticação (Token Ping)
<code>`claro_store`</code>	URL de marketplace gerada dinamicamente	Sim (a URL só é gerada para usuário)

	(login por MSISDN, já vem com `?origin=MINHA_CLARO_MOVELO`) <code>https://claro.com.br/marketplace/login/msisdn/</code>	autenticado)
`planos_celular`	`https://planoscelular.claro.com.br/`	Não
`claro_site`	`https://claro.com.br/`	Não

⚠ Estes são os **únicos 7 valores válidos**. Qualquer outro valor (erros de grafia, ou `type` ausente) faz o usuário cair direto na **tela de erro**.

Para os tipos marcados como "precisa de usuário logado": se o app não tiver o token necessário no momento do redirecionamento, o usuário também cai na tela de erro, mesmo que o resto da URL esteja correto.

⚠ Para que o redirecionamento logado para o site residencial ocorra é necessário que nos parâmetros da url sejam incluídos de forma obrigatória:

`&origin=minhanetapp`

`&tokenPing=true`

`&contractSession=true`

Confira o exemplo abaixo.

Parâmetros extras (query string repassada)

Qualquer parâmetro que **não esteja** na lista da seção 3 (`type`, `path`, `external`, `tokenAA`, `tokenPing`, `contractSession`) é automaticamente incluído como query string na URL final, sem nenhuma transformação de nome — só é feito o URL-encode do valor.

Isso permite montar URLs de destino com parâmetros específicos daquele site/sistema, por exemplo:

- `affiliateId=CeyxwiffV`
- `redirect-feature=ServicosConsultarPortabilidade`

- ``redirect=subscriptions&action=selection``
- ``utm_source=minhaclaro_app&utm_medium=plppos-clientepos&utm_campaign=Pos_abasGerenciar``
- ``q=:relevance:promotionalPaymentMethods:ticket``

Ou seja: use ``type`` e ``path`` para dizer "para onde" o usuário vai; use os demais parâmetros para dizer "com o que" ele chega lá" — eles são de responsabilidade de quem consome a página de destino (ex.: parâmetros do e-commerce, de campanha, de portabilidade, etc.), não da tela ``LoaderScreenGenericRedirect`` em si.

 Parâmetros repetidos são simplesmente repassados como estão — não há tratamento especial de arrays nem tratamento de digitação.

Regra de concatenação de ``path`` (atenção aqui!)

A URL final é montada assim:

```
1 | URL_final = URL_base_do_type + path + "?" + parâmetros_extras
```

A concatenação entre ``URL_base`` e ``path`` é **literal (string + string), sem inserir ``/`` automaticamente**. Como todas as URLs base já terminam com ``/`` (ex.: ``https://minhaclaroresidencial.claro.com.br/``), a regra prática é:

-  ``path`` **não deve começar com ``/``**

Exemplo correto: ``path=minhas-assinaturas`` →

```
`https://minhaclaroresidencial.claro.com.br/minhas-assinaturas`
```

-  Evite ``path=/minhas-assinaturas`` → geraria ``https://minhaclaroresidencial.claro.com.br//minhas-assinaturas`` (barra duplicada)

Se ``path`` já contiver seu próprio ``?parametro=valor``, isso também é preservado e mesclado corretamente com os demais parâmetros extras — mas o mais simples e recomendado é sempre colocar os parâmetros extras como parâmetros normais da URL do deep link, e não embutidos dentro do valor de ``path``.

Parâmetros injetados automaticamente (não precisa enviar)

Para alguns tipos, um parâmetro de token é adicionado **automaticamente** na URL final, sobrescrevendo qualquer valor manual enviado com o mesmo nome:

``type``

Parâmetro injetado

`flex`	`token=<token de autenticação do usuário>`
`claro_tv_mais`	`accessTokenId=<token de autenticação do usuário>`

- ❗ Não é necessário (nem recomendado) enviar esses parâmetros manualmente na URL — eles são preenchidos pelo app com o token da sessão atual do usuário.

💡 Exemplos comentados

```
1 minhaclaro://LoaderScreenGenericRedirect?type=residencial&path=minhas-as
```

Abre em WebView (`external` omitido = `false`) a página

`https://minhaclaroresidencial.claro.com.br/minhas-assinaturas?affiliateId=CeyxwiffV`, injetando cookies de `tokenAA` e `contractSession` (mas não `tokenPing`). Requer usuário logado (Token Ping).

```
1 minhaclaro://LoaderScreenGenericRedirect?type=flex&path=webview/minha-cl
```

Abre em WebView `https://flex.claro.com.br/webview/minha-claro/auth?token=<token>` (token injetado automaticamente). `tokenAA=true` por padrão. Requer usuário logado.

```
1 minhaclaro://LoaderScreenGenericRedirect?type=minha_claro&redirect-featu
```

Sem `path` → abre `https://minhaclaro.claro.com.br/?redirect-feature=ServicosConsultarPortabilidade` em WebView. Não requer login.

```
1 minhaclaro://LoaderScreenGenericRedirect?type=claro_store&external=true&
```

Abre no **navegador externo** (`external=true`) a URL de marketplace gerada dinamicamente, com `redirect=subscriptions&action=selection` anexados.

```
1 minhaclaro://LoaderScreenGenericRedirect?type=planos_celular&path=claro/
```

Exemplo de link de campanha: `path` aponta para uma página específica do catálogo e os `utm\_\*` são repassados como parâmetros extras de rastreamento.

```
1 minhaclaro://LoaderScreenGenericRedirect?type=claro_tv_mais&path=filme/s
```

Abre em WebView `https://www.clarotvmais.com.br/filme/super-mario-galaxy-o-filme/3576514?accessTokenId=<token>`, injetando o cookie de `tokenPing` e **não** injetando o de `tokenAA` (sobrescreve o padrão `true`). Requer usuário logado (Token Ping).

```
1 | minhaclaro://LoaderScreenGenericRedirect?type=blablablabla
```

``blablablabla`` **não é** um ``type`` válido (não existe na lista da seção 4) — este link sempre cai na tela de erro. Útil apenas para testar o fluxo de erro, não deve ser usado como link de produto.

✅ Checklist rápido para criar uma URL nova

1. Escolher um ``type`` válido da tabela da seção 4.
2. Confirmar se esse ``type`` exige usuário logado — se sim, o link só deve ser usado em contextos onde o usuário já está autenticado no app.
3. Definir ``path`` (sem ``/`` inicial) apontando para a página de destino dentro daquele domínio.
4. Definir ``external`` conforme o desejado (WebView dentro do app vs. navegador externo).
5. Ajustar ``tokenAA`` / ``tokenPing`` / ``contractSession`` somente se o padrão não servir (na maioria dos casos os padrões já são adequados).
6. Adicionar quaisquer parâmetros extras necessários pela página de destino (UTMs, filtros, IDs, etc.) — eles são passados como estão, sem necessidade de mapeamento.
7. Gerar as duas versões do link (deep link nativo e smart link) seguindo os formatos da seção 2.
8. Testar o link em ambiente de homologação antes de divulgar em campanha/produção.